Group project collaboration

Shared by raghav

This is a copy of a chat between Claude and raghav. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

Report

Files hidden in shared chats

working on a group project

Nice, PDEBench-Lang looks like a solid project! What do you need help with?

Firstly my job is data generation how should I go aobut doing this do not give me any code

Based on your proposal, here's how to think about the data generation pipeline:

What you're actually building

You need a dataset where each instance is one PDE equation that exists in all four dialects simultaneously, plus three ground-truth labels (structural reasoning, behavioral label, operator subset).

Step 1: Define your PDE families parametrically

For each of the five families (Heat, Wave, Burgers, Laplace, Advection), write down the general symbolic form with free coefficient slots. For example, Heat is always `u_t = α u_xx` — your only degree of freedom is α. Know exactly which coefficients are free, what ranges make physical sense, and whether any coefficient values would degenerate the equation into a different family (edge cases to avoid).

Step 2: Sample coefficients systematically

Decide on a sampling strategy per family — uniform, log-uniform (useful when coefficients span orders of magnitude), or a fixed grid. Make sure your ranges are physically meaningful and that you document them clearly so your teammates can reproduce everything exactly.

Step 3: Generate the canonical form first

Before any dialect conversion, establish one ground-truth symbolic representation (e.g., infix math notation). Everything else derives from this. This is your source of truth.

Step 4: Convert to all four dialects

- **Postfix** — mechanical tree traversal, fully algorithmic
- **Prefix** — same tree, different traversal
- **Raw LaTeX** — straightforward string templating from the canonical form
- **Natural Language** — the hardest one; you'll need to write templates per PDE family and per structural feature (e.g., how do you verbalize a mixed partial derivative?)

The natural language templates deserve the most care. Think about whether you want one canonical phrasing or slight paraphrase variation — variation is more realistic but harder to label consistently.

Step 5: Generate the three labels

- **Behavioral label** is just the family name — trivial since you know what you sampled
- **Operator subset** needs a decision: do you derive this analytically from the solution family (e.g., Heat solutions involve `exp` and polynomials) or from the equation structure itself? Clarify this with your team since it affects what Krivan is fine-tuning against
- **Structural reasoning chain** is the hardest label — you need templated, grammatically correct descriptions of derivative orders and equation structure that are *always factually correct*, since this is what the Trash Score is measured against

Step 6: Quality checks

- Verify no two instances are identical after rounding
- Check that no sampled coefficient creates a degenerate or ill-defined PDE
- Make sure dialect conversions are invertible (you should be able to reconstruct the canonical form from any dialect)
- Spot-check natural language outputs manually before handing the dataset off

Key decision to nail down with your team

How large does the dataset need to be per family? You're training four parallel

How large does the dataset need to be per family? You're training four parallel models, so you need enough instances that each dialect gets a fair training split. That number should be decided before you start sampling, not after.

should I do this algromathically or through ai

Do it algorithmically, without question.

Here's why:

Your ground-truth labels must be provably correct. The entire project hinges on the Trash Score — detecting when a model reasons incorrectly despite getting the right answer. If your labels were themselves generated by an LLM, you have no guarantee they're structurally accurate, which completely undermines that metric.

The dialect conversions are fully deterministic. Postfix, Prefix, and LaTeX are just tree traversals and string templates over a known symbolic structure. There's no ambiguity — an algorithm will be perfectly consistent every time. An LLM might paraphrase slightly differently across instances, introduce errors, or hallucinate operator names.

Reproducibility matters for a research project. Your teammates need to be able to regenerate the exact dataset with a fixed random seed. Algorithmic generation guarantees this. LLM generation does not.

You only have five PDE families with known structure. This isn't a case where the problem is too complex for rules — you can fully enumerate the structural features (derivative orders, operator types, equation class) for every instance analytically. There's no need to approximate with AI.

The one exception — Natural Language

This is the only dialect where pure templating gets awkward, because natural language has more surface variation. Even here though, the right approach is **hand-written templates per family** that you fill algorithmically, not LLM generation. You want controlled, consistent phrasing across instances, not creative variation that could confound your cross-dialect comparison.

Bottom line: AI generation would introduce exactly the kind of noise and unverifiability that would make your results hard to defend.

so I would I go about coding this even

Good question — let me break down the conceptual coding architecture before you write a single line.

The core abstraction: a PDE object

Everything starts with a data structure that represents one PDE instance. It should hold the family name, the sampled coefficients, and then all four dialect strings plus all three labels. Every other piece of code either produces this object or reads from it.

The three distinct modules you need

1. Sampler Takes a family name, samples coefficients from your defined ranges, and returns the canonical symbolic form. This is just random number generation with some bounds checking. One function per PDE family is cleanest.

2. Converter Takes the canonical form and produces all four dialect strings. This is where most of your work lives. Postfix and Prefix require you to think in terms of an expression tree — you build the tree from the canonical form, then traverse it differently for each dialect. LaTeX is string templating. Natural language is template filling with some conditional logic (e.g., "one-half" vs "0.5" depending on coefficient value).

3. Labeler Takes the canonical form and produces the three ground-truth labels. Behavioral label is just the family name. Operator subset is a hardcoded lookup per family — you know analytically that Heat solutions involve exp and polynomials, so this is just a dictionary. Structural reasoning is templated text describing derivative orders, filled in from the equation structure.

How they connect

```
Sampler → canonical form → Converter → four dialect strings
                         → Labeler  → three labels

All of it → one PDE object → write to disk
```

You run this in a loop N times per family, collect all objects, then serialize to JSON or CSV.

The expression tree question

This is the key technical decision. For Postfix and Prefix you need to traverse a tree. You have two options:

- **Build your own simple tree** — since your PDEs are structurally fixed (you're not parsing arbitrary equations, you're generating from known templates), you can hardcode the tree structure per family and just fill in coefficient values. Much

hardcode the tree structure per family and just fill in coefficient values. Much simpler.

- **Use a symbolic math library** like SymPy — lets you define the equation symbolically and then write a traversal function to emit Postfix/Prefix. More elegant and generalizes better, but adds a dependency and a learning curve.

Given you only have five families, hardcoding the trees is probably the pragmatic choice.

What to build first

Start with just the Heat equation end-to-end — one sampler, all four converters, all three labels, one output object. Get that fully working and verified before touching the other four families. The other families are then just additional instances of the same pattern.

Output format

JSON is the cleanest choice here. Each line is one PDE instance, all four dialects and all three labels present. Your teammates can load it directly for fine-tuning without any preprocessing.

What part of this do you want to dig into further?

where do you get canocial form

Good question — this is a conceptual point worth getting clear on.

You don't "get" it from anywhere — you define it yourself.

The canonical form is just whatever internal representation *you* decide is the source of truth before any dialect conversion happens. It's not a real mathematical standard, it's a design choice you make for your pipeline.

What it practically looks like

For your five families, the canonical form could be as simple as a Python dictionary like:

```
family: "Heat"
coefficients: {alpha: 0.5}
expression: "u_t = 0.5 * u_xx"
```

That's it. You constructed this — you didn't discover it. You sampled alpha from your

That's it. You constructed this — you didn't discover it. You sampled alpha from your defined range, plugged it into the known Heat equation template, and now you have a canonical object to convert from.

Why this works

Because you're not parsing arbitrary user-submitted equations. You are the one generating the equations from scratch. So you always know the structure — you just sampled it. The canonical form is just you recording what you sampled before converting it into dialects.

The flow clarified

```
Sample alpha = 0.5
→ Plug into Heat template: u_t = alpha * u_xx
→ This IS your canonical form
→ Now convert it to Postfix, LaTeX, Prefix, NL
```

There's no mystery upstream source. You are the source.

> so what are the equations we going to be using are there any libraries that do this for us

The five equations from your proposal:

Family	General Form
Heat (Diffusion)	`u_t = α u_xx`
Wave	`u_tt = c² u_xx`
Burgers	`u_t + u u_x = ν u_xx`
Laplace	`u_xx + u_yy = 0`
Advection	`u_t + c u_x = 0`

These are all fixed templates — your only freedom is sampling the coefficients (α, c, ν, etc.).

On libraries — yes, SymPy is the answer

SymPy is a Python symbolic math library and it does a surprising amount of your

SymPy is a Python symbolic math library and it does a surprising amount of your work:

- You define the equation symbolically using SymPy's syntax
- It builds the expression tree for you internally
- You can traverse that tree to emit Postfix and Prefix automatically
- It can output LaTeX directly with one function call (`sympy.latex()`)
- Coefficient substitution is built in

So instead of hardcoding trees yourself, SymPy handles the tree structure and you just write the traversal logic on top of it.

What SymPy does NOT do for you

- Natural language conversion — that's your templates
- Structural reasoning labels — your templates
- Operator subset labels — your hardcoded lookup
- Coefficient sampling — that's just Python's random module

Honest assessment

Given you have five fixed-template equations, SymPy is worth it mainly for the **LaTeX output and tree traversal.** It'll save you from writing fragile string manipulation code and make your Postfix/Prefix conversion much more reliable and generalizable.

Want to dig into how SymPy represents these equations internally?

> i do not undernstand these how are we genratiing this from random Natural language conversion — that's your templates
> * Structural reasoning labels — your templates
> * Operator subset labels — your hardcoded lookup
> * Coefficient sampling — that's just Python's random module when define equation any equation is it

Great question — let me untangle this completely.

The whole pipeline in plain english

You are not discovering equations. You are *manufacturing* them. Here's the full mental model:

Step 1: Pick a family and sample a coefficient

You tell Python "give me a random number between 0.1 and 2.0" — that becomes α. That's all coefficient sampling is. Nothing mathematical is happening yet.

```
α = 0.3  (randomly chosen)
```

Step 2: Plug it into the known template

You already know Heat is always `u_t = α u_xx`. You just fill in the blank.

```
u_t = 0.3 u_xx
```

This is your equation. You just built it. SymPy helps you represent this formally so you can manipulate it.

Now the four things you were confused about:

Coefficient sampling — already explained above. Just `random.uniform(0.1, 2.0)`. Done.

Operator subset — you hardcode this *per family* in advance because you already know the math. Heat equations always have solutions involving `exp` and polynomials — that's a known mathematical fact, not something you compute per instance. So your lookup is just:

```
Heat → {exp, polynomial}
Wave → {sin, cos, polynomial}
Burgers → {tanh, polynomial}
...
```

Same answer every time for the same family regardless of coefficient value.

Structural reasoning labels — you write a sentence template per family in advance:

```
Heat template:
"Contains a 1st-order time derivative and a
2nd-order spatial derivative. Structure



indicates diffusive transport. Coefficient
```

```
value is {α}."
```

Then you just fill in α. Every Heat instance gets this same template with its specific coefficient dropped in. You write five templates total, one per family.

Natural language conversion — same idea but for the equation itself, not the reasoning:

```
Heat NL template:
"The time derivative of u equals {α} times
the second spatial derivative of u."
```

Fill in α, done. Again five templates total.

So the full picture for one instance:

```
1. Sample α = 0.3
2. Build equation: u_t = 0.3 u_xx        ← SymPy
3. Convert to Postfix/Prefix/LaTeX        ← SymPy tree traversal
4. Fill NL template with 0.3             ← your template
5. Behavioral label = "Heat"             ← you just know this
6. Operator subset = {exp, polynomial}   ← your lookup table
7. Fill reasoning template with 0.3      ← your template
```

Steps 1 and 4-7 are trivially simple. Steps 2-3 are where SymPy earns its keep.

Does that make the pipeline feel more concrete?

why are we doing any of this why could'nt we just jus infex postlif latex just give it to the modle what is th point of this Fill NL template with 0.3 ← your template
5. Behavioral label = "Heat" ← you just know this
6. Operator subset = {exp, polynomial} ← your lookup table
7. Fill reasoning template with 0.3 ← your template

Great question — this gets at the heart of why your project exists.

What you're actually doing

You're not just converting equations between formats. You're training a model to do

You're not just converting equations between formats. You're training a model to do three things when it sees an equation:

1. Classify what type of PDE it is (behavioral label)
2. Predict which operators appear in the solution (operator subset)
3. Explain its reasoning correctly (structural reasoning)

The model needs to *learn* these things. That means during training it needs examples of inputs paired with correct answers. Those correct answers are exactly steps 4-7.

The analogy

Imagine teaching someone to recognize dog breeds. You can't just show them photos — you need photos *labeled* with the breed name. The labels are the supervision signal. Without them there's nothing to learn from.

Your labels are:

- The breed name → behavioral label
- Physical traits that indicate the breed → structural reasoning
- What the dog can do → operator subset

Why not just feed it equations with no labels?

You could, but then what is the model learning? There's no target to optimize toward. Fine-tuning requires input-output pairs. The outputs are your labels.

The actual research question

Once you train four parallel models (one per dialect) on these labeled examples, you compare them. Does the model trained on LaTeX input produce better reasoning than the one trained on Postfix input? That comparison is only possible because you have ground truth labels to measure against.

Without the labels there's no measurement. Without measurement there's no paper.

so not ionly are you give the modele infisx expression you arethen going to give it gehvaiou obperator subsent and reasoning as training ask to predict these at test right

Exactly, you got it.

Training time — you give it everything:

```
Input:  u_t = 0.3 u_xx        (one of the four dialects)
Output: Behavioral label    → "Heat"
        Operator subset     → {exp, polynomial}
        Structural reasoning → "Contains 1st order time
                                derivative and 2nd order
                                spatial derivative..."
```

The model sees input AND correct answers and learns the mapping.

Test time — you only give it the equation:

```
Input:  u_t = 0.7 u_xx        (unseen coefficients)
Output: model has to PREDICT all three on its own
```

Then you compare its predictions against your ground truth labels to measure performance.

And this is done four times in parallel:

One model trained on Postfix inputs, one on LaTeX, one on Prefix, one on Natural Language — all trained on the same underlying equations, just different surface representations.

Then at test time you ask:

- Which dialect model predicted the operator subset most accurately?
- Which dialect model had the lowest Trash Score?
- Which dialect model classified PDE families best?

That cross-dialect comparison IS your research contribution. You're not building one model — you're running a controlled experiment across four models where the only variable is the input representation.

for the natural langue how can that be generated we can generate infex prefix and post fix using just one equation however natural language how

Great point — natural language is fundamentally different from the other three.

Postfix, Prefix, LaTeX are all just mechanical transformations of the same expression tree. One equation, deterministic output every time.

Natural language is not a transformation — it's a description. You can't algorithmically derive "the time derivative of u equals one-half the second spatial derivative of u" from a tree traversal the same way.

But here's why it's still manageable

You only have five PDE families. And crucially, the *structure* of each family never changes — only the coefficients do. So you only need to write one template per family.

Heat always looks like:

```
u_t = α u_xx
```

So the natural language always looks like:

```
"The time derivative of u equals {α} times
the second spatial derivative of u."
```

The only thing that changes instance to instance is the coefficient value. Everything else is fixed. So you write five sentences total, hardcode them, and fill in the number programmatically.

The one wrinkle — coefficient phrasing

You probably want some basic prettification logic:

- 0.5 → "one-half"
- 1.0 → just drop it entirely ("the second spatial derivative" not "1.0 times")
- 2.0 → "twice"
- everything else → just the number

That's a small lookup, not complex logic.

What about Burgers — it has no free coefficient in the nonlinear term

What about Burgers — it has no free coefficient in the nonlinear term

```
u_t + u u_x = ν u_xx
```

Here ν is your only free coefficient. Natural language becomes:

```
"The time derivative of u plus u times its
spatial derivative equals {ν} times the second
spatial derivative of u."
```

Still just one template, one coefficient to fill in.

Bottom line

Natural language generation for your project is just five handwritten sentence templates with one number substituted in. It feels scarier than it is because "natural language generation" sounds like a hard AI problem — but given your constrained setup it's actually the simplest part of the pipeline.

now give whole pipeline for data generation

The Complete Data Generation Pipeline

Phase 1: Define your PDE templates

Before writing any code, you hardcode the structure of all five families. For each family you define:

- The general symbolic form
- Which coefficients are free
- The sampling range for each coefficient
- The natural language template
- The structural reasoning template
- The operator subset (fixed per family)

This is all static knowledge that never changes. You write it once.

Phase 2: Coefficient Sampling

Phase 2: Coefficient Sampling

For each instance:

1. Pick a family (you can cycle through all five evenly or sample randomly)
2. Sample each free coefficient from its defined range using `random.uniform`
3. Apply prettification logic — round to 2 decimal places, avoid degenerate values (e.g. $\alpha = 0$ turns Heat into a trivial equation, reject and resample)
4. Store the coefficient values

Phase 3: Build the Canonical Form via SymPy

1. Define your PDE symbolically in SymPy using the sampled coefficients
2. This gives you a SymPy expression object — your internal source of truth
3. Everything downstream derives from this object

Phase 4: Dialect Conversion

From the SymPy expression object, generate all four dialects in parallel:

LaTeX

- Call SymPy's built-in latex() function
- Minor cleanup if needed for readability
- Done

Prefix

- Write a recursive tree traversal function
- At each node, emit the operator first then the operands
- Operands are traversed left to right
- Leaf nodes (coefficients, variables) emit their value directly

Postfix

- Same recursive traversal
- At each node, emit operands first then the operator
- Mirror of Prefix logic

Natural Language

- Look up the template for the current family

- Look up the template for the current family
- Substitute coefficient values
- Apply prettification (0.5 → "one-half", 1.0 → drop it, etc.)
- Output the filled string

Phase 5: Label Generation

Three labels, all derived from what you already know:

Behavioral Label

- Trivial — it's just the family name you already have
- "Heat", "Wave", "Burgers", "Laplace", "Advection"

Operator Subset

- Look up your hardcoded dictionary for the current family
- Heat → {exp, polynomial}
- Wave → {sin, cos, polynomial}
- Burgers → {tanh, polynomial}
- Laplace → {sin, cos, exp, polynomial}
- Advection → {exp, polynomial}
- No computation needed

Structural Reasoning

- Look up the template for the current family
- Fill in derivative orders, coefficient values
- This must be factually precise — it is your ground truth for the Trash Score

---

Phase 6: Assemble the Instance Object

Bundle everything into one structured object per instance:

```
{
  family: "Heat",
  coefficients: {alpha: 0.3},
  dialects: {
    latex:    "...",
    prefix:   "...",
    postfix:  "...",
```

```
    natural:  "..."
  },
  labels: {
    behavioral:  "Heat",
    operators:   ["exp", "polynomial"],
    reasoning:   "..."
  }
}
```

Phase 7: Dataset Assembly

1. Decide total dataset size — recommend at least 1000 instances per family, 5000 total minimum given you're training four parallel models
2. Run the pipeline in a loop
3. After each instance, validate it — check no degenerate coefficients, check all four dialects generated successfully, check labels populated
4. Reject and resample any failed instances
5. Shuffle the full dataset
6. Split into train / validation / test (e.g. 70/15/15)
7. Make sure the split is stratified — equal family representation in each split

---

Phase 8: Serialize to Disk

1. Write to JSON lines format — one instance per line
2. Write three files: `train.jsonl`, `val.jsonl`, `test.jsonl`
3. Save a separate metadata file recording your sampling ranges, random seed, dataset size per family, and split sizes
4. The random seed is critical — record it so the dataset is fully reproducible

---

The full flow visually

```
For each instance:
|
├── Sample coefficients (random.uniform)
|
├── Build SymPy expression (canonical form)

|
```

```
│
├── Convert to four dialects
│   ├── LaTeX     ← sympy.latex()
│   ├── Prefix    ← recursive tree traversal
│   ├── Postfix   ← recursive tree traversal
│   └── Natural   ← template + coefficient fill
│
├── Generate three labels
│   ├── Behavioral  ← family name
│   ├── Operators   ← hardcoded lookup
│   └── Reasoning   ← template + coefficient fill
│
├── Validate instance
│
└── Write to disk
```

What you hand off to your te

Start your own conversation